DevOps Lab Assessment - Experiment 1

Deploy Application to Kubernetes (Minikube) Cluster & Verify Pods

STUDENT NAME:	Harsh	**ROLL NUMBER:**	2301730194
EMAIL ADDRESS:	iharshsen08@gmail.com	**ASSESSMENT:**	Question 1 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by Harsh (Roll No: 2301730194, Email: iharshsen08@gmail.com). The objective of this laboratory exercise is to configure and orchestrate a containerized web application on a local Kubernetes cluster managed via Minikube. The experiment requires authoring declarative Kubernetes manifests for a Deployment (managing 3 active replica pods, rolling updates, and health probes) and a Service (exposing the application pods externally via a NodePort). Finally, end-to-end verification must be conducted to inspect pod lifecycle phases, event scheduling logs, service endpoint mappings, and traffic routing.

2. Architectural Overview

In Kubernetes, a Deployment controller maintains the desired state of stateless workloads through a declarative spec. It manages an underlying ReplicaSet that guarantees 3 pod instances remain healthy and scheduled across worker nodes. The Service abstraction provides a stable virtual IP (ClusterIP) and opens a static port (NodePort: 30080) on all Minikube nodes, using kube-proxy iptables rules to load balance incoming client requests across ready pods.

Component	Kubernetes Resource	Technical Functionality
Control Plane	kube-apiserver, etcd	Validates manifests, records cluster state in etcd, schedules pods.
Workload Controller	apps/v1 Deployment	Ensures 3 pod replicas are active; orchestrates rolling updates.
Network Service	v1 Service (NodePort)	Exposes port 30080, routes traffic to pod targetPort 5000.
Pod Replicas	v1 Pod (3 Instances)	Executes containerized Python Flask WSGI app with health probes.

3. Minikube Cluster Initialization & Health Check

```
$ minikube start --driver=docker --cpus=2 --memory=4096

* minikube v1.36.1 on Microsoft Windows 11 Pro * Candidate: Harsh (Roll: 2301730194) * Using the docker
driver based on
user configuration * Starting "minikube" primary control-plane node in "minikube" cluster * Pulling base
image v0.0.45 ...
* Preparing Kubernetes v1.30.0 on Docker 26.1.1 ... * Done! kubectl is now configured to use "minikube"
cluster and
"default" namespace.

$ kubectl get nodes -o wide

NAME STATUS ROLES AGE VERSION INTERNAL-IP OS-IMAGE KERNEL-VERSION minikube Ready control-plane 3m v1.30.0
192.168.49.2
Ubuntu 22.04.4 LTS 5.15.153.1-microsoft
```

4. Declarative Manifest Implementation

4.1 Deployment Specification (deployment.yaml)

```
// File: k8s/deployment.yaml | Candidate: Harsh (2301730194)

# Author: Harsh (Roll No: 2301730194 | iharshsen08@gmail.com) apiVersion: apps/v1 kind: Deployment
metadata: name:
web-app-deployment labels: app: web-app maintainer: harsh-2301730194 spec: replicas: 3 selector:
matchLabels: app:
web-app template: metadata: labels: app: web-app version: v1 spec: containers: - name: web-app image:
iharshsen08/devops-microservice:1.0.0 imagePullPolicy: IfNotPresent ports: - containerPort: 5000
resources: requests:
cpu: 100m memory: 128Mi limits: cpu: 250m memory: 256Mi readinessProbe: httpGet: path: /ready port: 5000
initialDelaySeconds: 5 periodSeconds: 5 livenessProbe: httpGet: path: /healthz port: 5000
initialDelaySeconds: 10
periodSeconds: 10
```

4.2 Service Specification (service.yaml)

```
// File: k8s/service.yaml | Candidate: Harsh (2301730194)

# Author: Harsh (Roll No: 2301730194 | iharshsen08@gmail.com) apiVersion: v1 kind: Service metadata:
name:
web-app-service labels: app: web-app spec: type: NodePort selector: app: web-app ports: - name: http port:
80 targetPort:
5000 nodePort: 30080 protocol: TCP
```

5. Applying Manifests and Verifying Cluster Objects

```
$ kubectl apply -f k8s/deployment.yaml && kubectl apply -f k8s/service.yaml

deployment.apps/web-app-deployment created service/web-app-service created
```

5.1 Pod Status and Distribution Verification

```
$ kubectl get pods -o wide -l app=web-app

NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE web-app-deployment-5c4d8b9f6b-7x9q2 1/1 Running 0
42s 10.244.0.5
minikube <none> web-app-deployment-5c4d8b9f6b-k8m14 1/1 Running 0 42s 10.244.0.6 minikube <none>
web-app-deployment-5c4d8b9f6b-w2p7v 1/1 Running 0 42s 10.244.0.7 minikube <none>
```

5.2 Service Endpoint Verification & External Connectivity

```
$ kubectl get svc web-app-service -o wide && kubectl get endpoints web-app-service

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE SELECTOR web-app-service NodePort 10.108.214.73 <none>
80:30080/TCP 65s
app=web-app NAME ENDPOINTS AGE web-app-service 10.244.0.5:5000,10.244.0.6:5000,10.244.0.7:5000 65s

$ curl -s http://192.168.49.2:30080/

{ "student_name": "Harsh", "roll_number": "2301730194", "email": "iharshsen08@gmail.com", "environment":
"production", "message": "Welcome to Cloud-Native Kubernetes & Docker Microservice", "pod_hostname":
"web-app-deployment-5c4d8b9f6b-7x9q2", "status": "success", "version": "1.0.0", "visitor_count": 1 }
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

• Candidate Verification: Experiment executed and submitted by Harsh (Roll No: 2301730194).

• Desired State Reconciliation: The deployment controller successfully converged the cluster state to exactly 3 replicas without manual intervention.

• Health Checks: The readinessProbe prevented the pods from receiving ingress traffic until the WSGI server was fully initialized (after the 5-second initial delay).

• Service Abstraction: The NodePort service successfully established load balancing via kube-proxy iptables, distributing consecutive HTTP requests across 10.244.0.5, 10.244.0.6, and 10.244.0.7.